```
server.listen '8080', 'localhost'
```

In production this might need to be something else:

```
server.listen '80, 'agtronsblog.com'
```

This is just one example of something that's likely to change when you run the application on another environment, such as the production environment.

Which environment is this?

The application doesn't magically know which environment it's running in. You have to tell it by using an *environment variable*. Inside a Node.js application, you can access environment variables through `process.env`. The environment variable you'll use is called `NODE_ENV` and is either set for you by the environment you're running on or is set by you when your program is started on the command line. Suppose it's already set; *here's how not to use the `NODE_ENV` environment variable*:

```
if process.env.NODE_ENV is 'production'
  server.listen '80', 'agtronsblog.com'
else if process.env.NODE_ENV is 'development'
  server.listen '8080', 'localhost'
```

Don't do this everywhere in your program; it would be a maintenance nightmare later down the track.

Instead of using `NODE_ENV` in many parts of your program, you should read it once and keep all of the environment information in a single place. That's what a manifest, or *environment config*, does. So how do you load one?

Loading an environment config

The application needs to know how to run on the environment it's deployed to. This information is provided in the manifest.

Using Node.js on Windows?

Some of the listings in this chapter won't work as is on all operating systems (such as Windows). They're designed to run on Unix-like operating